AI Code Review Tool — Team Task Board

Project: AI Code Review Tool (MERN + Groq API)
Version: v1.0 | April 2026
Team: 4 Teammates, 4 Tasks Each
Branch Rule: Each teammate works on their own branch and submits a PR for review.

Teammate 1 — UI Polish

Branch prefix: `feature/ui-polish-*`
Files: `frontend/src/App.css`, `frontend/src/components/CodeInput.jsx`

Task 1A — Fix the textarea: make it easier to write code in

What to do: The textarea feels cramped right now. Make it look like a proper code editor area.

Steps:

1. Open `App.css` and find the textarea style
2. Set `font-family` to `'Fira Code', monospace` (already imported in the CSS)
3. Set `font-size` to `14px`, `line-height` to `1.6`, `padding` to `14px`
4. Set `min-height` to `200px` so there's enough room to paste code
5. Set `resize` to `vertical` so the user can drag it taller if needed

Files to edit: `App.css` only
PR branch: `feature/ui-polish-textarea`

Task 1B — Make the submit button look and feel better

What to do: The button works but has no visual feedback when you hover or click it. Fix that.

Steps:

1. In `App.css`, find the `.btn-primary` style
2. Add `transition: background 0.2s ease` so color changes smoothly
3. Add a `:hover` rule that darkens the background slightly (use `var(--primary-hover)`)
4. Add an `:active` rule with `transform: scale(0.98)` so it feels "pressed"
5. Add `cursor: pointer` so the cursor changes on hover

Files to edit: `App.css` only
PR branch: `feature/ui-polish-button`

Task 1C — Replace language text input with a dropdown

What to do: Right now users can type anything in the language field. A dropdown is cleaner and prevents typos.

Steps:

1. Open `CodeInput.jsx` and find the `<input>` for language
2. Replace it with a `<select>` element with the same `value` and `onChange` props
3. Add these `<option>` values: `python`, `javascript`, `typescript`, `java`, `cpp`, `go`, `rust`, `plaintext`
4. Display them as: Python, JavaScript, TypeScript, Java, C++, Go, Rust, Plaintext
5. Style the select in `App.css` to match the textarea (same border, padding, border-radius)

Files to edit: `CodeInput.jsx` + `App.css`
PR branch: `feature/ui-polish-dropdown`

Task 1D — Add a character counter below the textarea

What to do: Users don't know there's a 5000 character limit until they hit it. Show them a live count.

Steps:

1. In `CodeInput.jsx`, below the textarea, add a `<p>` tag showing: `{code.length} / 5000`
2. Style it small and right-aligned (`font-size: 12px`, `text-align: right`, `color: var(--text-muted)`)
3. When `code.length` goes above 4000, change the color to `var(--warning)` (amber)
4. When `code.length` goes above 4900, change the color to `var(--danger)` (red)
5. Tip: use a simple inline style or a ternary for the color change

Files to edit: `CodeInput.jsx` + `App.css`
PR branch: `feature/ui-polish-char-count`

Teammate 2 — Output Display

Branch prefix: `feature/output-*`
Files: `frontend/src/components/ReviewResult.jsx`, `frontend/src/components/IssueList.jsx`, `frontend/src/App.jsx`, `frontend/src/App.css`

Task 2A — Show the score clearly at the top of results

What to do: The score badge exists but is plain text. Make it visually clear — users should see it immediately.

Steps:

1. Open `ReviewResult.jsx` and find the `score-badge` div
2. Make it bigger: `font-size: 1.2rem`, `padding: 10px 20px`, `border-radius: 8px`
3. The score class logic already exists (`score-high`, `score-mid`, `score-low`) — just improve the CSS colors
4. In `App.css`: `score-high` = green background, `score-mid` = amber, `score-low` = red
5. Add a small label above it saying "Code Quality Score" in muted text

Files to edit: `ReviewResult.jsx` + `App.css`
PR branch: `feature/output-score-badge`

Task 2B — Show a "No issues found" message for empty sections

What to do: Right now if `bugs=[]` the bugs section just disappears silently. That's confusing — users think the AI skipped it.

Steps:

1. Open `IssueList.jsx` and find the `if (!items || items.length === 0) return null` line
2. Instead of returning `null`, return a small message: "No issues found in this category"
3. Style it with muted text color and a checkmark character (✓) before it
4. Still show the section title above it so users know that category was checked
5. This applies to all 4 categories: bugs, issues, improvements, performance

Files to edit: `IssueList.jsx` + `App.css`
PR branch: `feature/output-empty-state`

Task 2C — Add a copy button to the refactored code block

What to do: The refactored code is shown in a code block but users have to manually select and copy it. Add a button.

Steps:

1. In `ReviewResult.jsx`, find the `refactored-code-container` div
2. Add a `<button>` inside it, positioned top-right (`position: absolute`, `top: 8px`, `right: 8px`)
3. On click: call `navigator.clipboard.writeText(suggestions.refactored_code)`
4. Use `useState` to track copied state — show "Copy" normally, "Copied!" for 2 seconds after clicking
5. Make the `refactored-code-container` `position: relative` so the button sits inside it correctly

Files to edit: `ReviewResult.jsx` + `App.css`
PR branch: `feature/output-copy-button`

Task 2D — Auto-scroll to results and add section dividers

What to do: When all sections are visible the page gets very long. Add visual dividers and auto-scroll after submit.

Steps:

1. In `App.css`, add a bottom border or margin to each `.issue-section` so sections feel separated
2. Wrap the `<ReviewResult />` component in `App.jsx` in a div with `id="result-section"`
3. After `setReviewData(response.data)`, add: `document.getElementById('result-section')?.scrollIntoView({ behavior: 'smooth' })`
4. This makes the page automatically scroll down to show results after submission
5. Test it: submit code, confirm the page smoothly scrolls to the result

Files to edit: `App.jsx` + `App.css`
PR branch: `feature/output-scroll-dividers`

Teammate 3 — History Page

Branch prefix: `feature/history-*`
Files: `frontend/src/pages/HistoryPage.jsx` (new), `frontend/src/main.jsx`, `frontend/src/App.jsx`, `frontend/src/components/Header.jsx`

Task 3A — Install React Router and set up two routes

What to do: Before building the history page, you need React Router so the app can have multiple pages.

Steps:

1. Run: `npm install react-router-dom` inside the `frontend/` folder
2. Open `main.jsx` and wrap `<App />` with `<BrowserRouter>` from `react-router-dom`
3. In `App.jsx`, import `Routes`, `Route` from `react-router-dom`
4. Add two routes: `<Route path="/" element={<MainPage />} />` and `<Route path="/history" element={<HistoryPage />} />`
5. Move the current App content into a new file `src/pages/MainPage.jsx` — this keeps things organized

Files to edit: `main.jsx`, `App.jsx`, new `src/pages/MainPage.jsx`
PR branch: `feature/history-router-setup`

Task 3B — Add a simple nav bar with Home and History links

What to do: Users need a way to get to the history page. Add a small nav bar to the top.

Steps:

1. Open `Header.jsx` and add two links below the title
2. Import `NavLink` from `react-router-dom`
3. Add: `<NavLink to="/">Home</NavLink>` and `<NavLink to="/history">History</NavLink>`
4. Style them in `App.css` — small, horizontal, spaced apart, with an underline on the active link
5. `NavLink` automatically adds an `active` class to the current route — use that for the underline style

Files to edit: `Header.jsx` + `App.css`
PR branch: `feature/history-navbar`

Task 3C — Build the history page: fetch and list past reviews

What to do: Create a new page that shows all past reviews fetched from the backend.

Steps:

1. Create `src/pages/HistoryPage.jsx`
2. On load (`useEffect`), call `axios.get('/api/reviews')` and save the result to a `useState` array
3. Show a loading message while fetching, and an error message if it fails
4. For each review in the list, show: language (as a badge), the score, and the date formatted nicely
5. Format date like this: `new Date(review.createdAt).toLocaleDateString()` — simple and clean

Files to edit: New `src/pages/HistoryPage.jsx`
PR branch: `feature/history-page`

Task 3D — Make each history item expandable to show full review

What to do: When a user clicks a past review in the list, show the full suggestions below it.

Steps:

1. Add a `useState` to track which review is expanded: `const [openId, setOpenId] = useState(null)`
2. When a row is clicked, set `openId` to that review's `_id` (or `null` to collapse)
3. Below each row, conditionally render the suggestions if `openId === review._id`
4. You can reuse the existing `IssueList` component for bugs, issues, improvements, performance
5. Add a small arrow (`>`) that rotates to `v` when expanded using CSS `transform`

Files to edit: `HistoryPage.jsx` + `App.css`
PR branch: `feature/history-expandable`

Teammate 4 — Validation & Error Handling

Branch prefix: `feature/validation-*`
Files: `frontend/src/App.jsx`, `frontend/src/components/CodeInput.jsx`, `frontend/src/App.css`

Task 4A — Show a clear error if the code box is empty on submit

What to do: Right now if you submit empty code, the error comes from the backend. Catch it earlier on the frontend.

Steps:

1. In `CodeInput.jsx`, before calling the API, check if `code.trim()` is empty
2. If empty, set a local error message: "Please paste some code before submitting"
3. Show this message in red text directly below the textarea — not in the global error box at the top
4. Also check if `code.length < 5` and show: "Code is too short to review"
5. Clear the error message as soon as the user starts typing again (use `onChange` to reset it)

Files to edit: `CodeInput.jsx` + `App.css`
PR branch: `feature/validation-empty-check`

Task 4B — Disable the button and show a loading state during the API call

What to do: Right now you can click the button multiple times while it's loading. Disable it and show feedback.

Steps:

1. The button already has `disabled={isLoading}` — verify this is working correctly
2. Update the button text: when `isLoading` is true show "Analyzing..." instead of "Review Code"
3. Add a simple CSS spinner inside the button when loading — a small rotating border circle using `@keyframes`
4. Add `opacity: 0.7` and `cursor: not-allowed` to the disabled button style in `App.css`
5. Make sure `isLoading` is set to `true` before the API call and `false` in the `finally` block — check `App.jsx`

Files to edit: `App.jsx` + `App.css`
PR branch: `feature/validation-loading-state`

Task 4C — Handle API errors properly and show useful messages

What to do: Right now all errors show the same message. Different errors should give different guidance.

Steps:

1. In `App.jsx` inside the `catch` block, check `err.response?.status`
2. If status is `400`: show "Invalid input — check your code and try again"
3. If status is `429`: show "Too many requests — please wait a moment before trying again"
4. If status is `500` or undefined: show "Something went wrong on the server — try again in a few seconds"
5. Style the error box in `App.css` with a left red border (`border-left: 3px solid var(--danger)`) to make it stand out

Files to edit: `App.jsx` + `App.css`
PR branch: `feature/validation-error-messages`

Task 4D — Clear old results when a new submission starts

What to do: Right now old results stay visible while new ones are loading. That's confusing — clear them immediately.

Steps:

1. In `App.jsx`, inside `handleReviewSubmit`, at the very top set: `setReviewData(null)` and `setErrorMessage('')`
2. This means as soon as submit is clicked, the old result disappears
3. Also reset the result when the user edits the code: in the textarea's `onChange`, call `setReviewData(null)`
4. This way users always know the visible result matches the current code
5. Test it: submit, wait for result, edit code slightly — the result should disappear immediately

Files to edit: `App.jsx` only
PR branch: `feature/validation-clear-on-edit`

PR Workflow

1. Pick your tasks above
2. Create your branch: `git checkout -b feature/your-branch-name`
3. Make changes, commit with a clear message
4. Push and open a Pull Request to `main`
5. Tag the repo owner for review

> **Note for Teammate 3:** Submit Task 3A (router setup) as your first PR — Teammate 4 should wait for it to merge before finalizing any `App.jsx` changes.